

STRATA

AI-driven source separation · Real-time 3D music visualisation

Status	Deployed ✓	Date	June 7, 2026	URL	ecenazg.github.io/strata
--------	------------	------	--------------	-----	---

What Is Strata?

Strata is an AI-powered real-time 3D music visualizer. It separates a song into its four hidden instrument layers — **bass, drums, melody, and harmony** — and translates each layer into a distinct cinematic 3D animation, synchronized to the audio as it plays in the browser.

The name refers to geological layers: music has hidden strata too, and this project makes them visible.

Layer	Color	Visualizer	Visual Effect
Bass	Blue #0b7cff	BassShockwave	Torus rings expand on every bass hit, floor glow pulses
Drums	Gold #ffd28a	DrumParticles	Particle bursts fire on each drum transient onset
Melody	Green #6dff8f	MelodyRibbon	Flowing 3D ribbon colored by spectral brightness, height by pitch
Harmony	Purple #ba7cff	HarmonicCloud	Luminous floating cloud shaped by harmonic energy

Python Backend — Analysis Pipeline

The backend transforms a raw MP3 into a 60 fps JSON feature stream that drives the entire frontend. It runs offline — no server is needed at runtime.

Step 1 — Source Separation (Demucs)

The song is processed by Meta's **htdemucs** model, a state-of-the-art hybrid transformer architecture that operates in both the waveform and spectrogram domains simultaneously. It separates the audio into four stems:

- vocals → melody
- bass → bass
- drums → drums
- other → harmonic

Separated .wav files are cached in stems/ for reuse.

Step 2 — Feature Extraction (librosa)

Each stem is analyzed at 44,100 Hz with a 512-sample hop (~11.6 ms resolution). Six features are extracted per frame:

Feature	Range	What it captures	Used by
rms	0 – 1	Volume / energy level (normalized)	All layers
onset	0 – 1	Attack sharpness — how sudden a sound hits	Drums, beat pulse
spectral_centroid	Hz	Brightness — where frequency mass is centered	Melody ribbon color
chroma	12 × [0,1]	Per-semitone pitch energy (one entry per note)	Melody pitch, harmonic cloud
pitch_hz	Hz	Dominant pitch detected via YIN algorithm	Melody ribbon height
beat_phase	0 – 1	Position within the current beat cycle	Camera punch, bloom

Step 3 — Down-sampling to 60 FPS

Raw librosa output is ~86 fps. Every ~1.43 frames we emit one output frame, producing exactly 60 fps to match the Three.js render loop. The result is saved as `features.json` and bundled with the frontend.

Key Data Structures

Class	Description
StemFeatures	Per-frame features for one instrument stem (rms, onset, centroid, chroma, pitch, beat_phase)
FrameFeatures	All four stems at one timestamp (t, frame_idx, melody, bass, drums, harmonic)
FeatureTimeline	Complete song analysis — audio path, duration, sample rate, fps, and all FrameFeatures; serializes to/from JSON

JavaScript Frontend — Real-Time 3D Visualizer

Stack: **Three.js**, **Vite**, **lil-gui**. The frontend loads `features.json` and the MP3, then drives a full 3D WebGL scene synchronized to the audio.

AudioSyncManager

- Loads the feature timeline from JSON on startup
- Plays the HTML5 `<audio>` element and runs a `requestAnimationFrame` loop
- On each frame, finds the correct `FrameFeatures` by binary-searching the timestamp
- Dispatches feature data to all four visualizers at 60 fps
- Supports play, pause, and seek — including click-to-drag on the progress bar

SceneManager

Owns the Three.js scene, camera, lighting, and post-processing pipeline:

- **Camera:** Perspective camera with drag-to-orbit and scroll-to-zoom. Auto-rotates slowly when idle. Beat-pulse pushes the camera forward for a punch effect.
- **Lighting:** Ambient fill + directional key light + warm point light + cyan point light. Warm and cyan intensities pulse with bass and harmonic energy in real time.
- **Post-processing:** UnrealBloomPass — bloom strength scales with beat energy for a glowing HDR look.
- **Star field:** 900 colored particles forming a deep-space backdrop, slowly rotating.
- **Energy rings:** 4 concentric torus rings at the bass layer origin, pulsing outward on bass hits.
- **Separation streams:** Animated Catmull-Rom spline streams that visualize the AI signal flowing from the source into each separated stem.

Four Visualizers

Visualizer	File	What it shows	Three.js primitive
● BassShockwave	BassShockwave.js	Bass energy as expanding floor rings + shockwave	Torus + CircleGeometry
● MelodyRibbon	MelodyRibbon.js	Melody as a flowing 3D ribbon colored by pitch	BufferGeometry line + vertex colors
● DrumParticles	DrumParticles.js	Drum hits as exploding particle bursts	Points / particle system
● HarmonicCloud	HarmonicCloud.js	Harmonic field as a floating luminous cloud	Instanced mesh / point cloud

Phase Narrative System

The song is divided into 8 cinematic phases that spotlight one layer at a time, telling the story of AI source separation:

Time	Phase	Description
0 – 4s	Opening	All layers faint — tension builds
4 – 8s	Source Separation	Signal streams animate apart visibly
8 – 16s	Bass Layer	Bass at full mix, others dimmed
16 – 24s	Drum Transients	Drums spotlight, particles surge
24 – 32s	Melody Ribbon	Melody ribbon glows at full intensity
32 – 40s	Harmonic Field	Cloud expands to fill the space
40s+	Combined Strata	All four layers at full power
Final 8s	Closing	Outro lockup fades in

Cinematic Overlay

- **Title lockup:** "STRATA" headline with eyebrow text and subtitle

- **Phase label:** Live text showing the current narrative phase
- **Stem legend:** Color-coded guide for all four instrument layers
- **Progress bar:** Clickable / draggable seek bar with ARIA accessibility attributes
- **Closing sequence:** Fade-in outro text when the song nears its end

Debug Panel (press G) + Video Recorder (press R)

- Toggle each of the 4 instrument layers on/off individually via lil-gui
- Built-in canvas recorder: captures a .webm video at 60 fps, 7 Mbps — download triggers automatically on stop

Architecture

[illegible]

v

WebGL Canvas → Browser

Deployed → <https://ecenazg.github.io/strata/>

Tech Stack

Layer	Technology	Purpose
Separation	Meta Demucs htdemucs	State-of-the-art hybrid transformer source separation
Analysis	librosa + NumPy	Audio feature extraction: RMS, onset, chroma, pitch, beat
Data format	Python dataclasses + JSON	Typed feature contract between backend and frontend
3D engine	Three.js (WebGL)	Scene, visualizers, post-processing, camera controls
Build tool	Vite	Fast HMR dev server, optimized static production build
Debug UI	lil-gui	Layer toggles, video recorder (canvas captureStream API)
Hosting	GitHub Pages	Zero-server static deployment, automatic on git push

Progress Summary

Completed

- ✓ Python source separation pipeline (Demucs)
- ✓ librosa feature extraction (6 features × 4 stems)
- ✓ 60 fps down-sampling to features.json
- ✓ AudioSyncManager (play / pause / seek)
- ✓ BassShockwave visualizer
- ✓ MelodyRibbon visualizer
- ✓ DrumParticles visualizer
- ✓ HarmonicCloud visualizer
- ✓ 8-phase narrative system
- ✓ Cinematic overlay + progress bar
- ✓ Drag-to-orbit camera + beat punch
- ✓ UnrealBloomPass post-processing
- ✓ Built-in canvas video recorder
- ✓ GitHub Pages deployment

Next Steps

- Upload any song (drag & drop)
- WebSocket real-time backend mode
- Waveform oscilloscope visualizer
- Frequency spectrum bar visualizer
- Chord / key detection display
- Server-side MP4 export (ffmpeg)
- Mobile touch optimisation
- Multiple song library support

Live Demo

<https://ecenazg.github.io/strata/>

Built with Python (Demucs, librosa, NumPy) · Three.js · Vite · GitHub Pages